

Base de Datos Avanzada

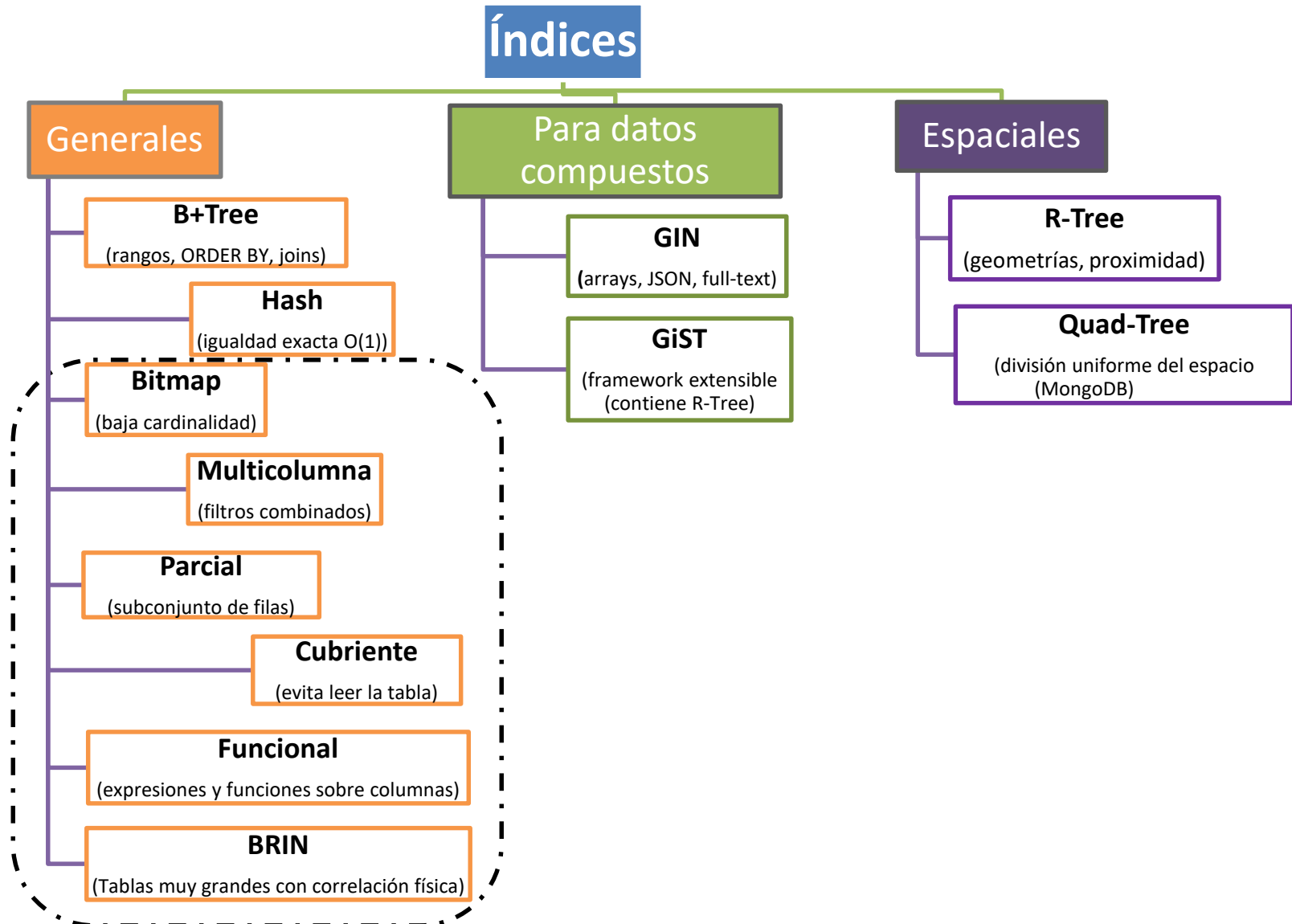


Profesores: Ricardo Arroyo
Jorge Pérez Herrera[©]

Índices en Base de Datos

Índices generales (Parte II)

Índices en Base de Datos



Índices en Base de Datos

Índices Bitmap

Índices en Base de Datos

Índices Bitmap

Características

- Bits en lugar de punteros.
- Ideal para columnas de baja cardinalidad (género, estado civil o activo/inactivo)
- Permiten acelerar consultas que combinan múltiples filtros con operaciones lógicas (**AND, OR**)
- No recomendables en tablas con muchas **INSERT/UPDATE**
 - mantenimiento puede ser costoso
 - bloqueo el bitmap entero.

Índices en Base de Datos

Índices Bitmap

Estructura

- Representan cada valor posible de una columna como un vector de bits.

Funcionamiento

- Cada fila corresponde a un bit
 - **1** indica presencia del valor y **0** ausencia.

Limitaciones

- Disponibilidad limitada
 - No todos los motores SQL soportan índices Bitmap (Oracle sí, PostgreSQL no de forma nativa).
- No sirven bien en alta cardinalidad

Índices en Base de Datos

Índices Bitmap

Tabla: clientes

ID	estado
1	activo
2	inactivo
3	activo
4	suspendido
5	activo
6	inactivo
7	activo

índice →



Bitmap por cada valor distinto
1 bit por fila (1 tiene ese valor; 0 No)

	F1	F2	F3	F4	F5	F6	F7
activo	1	0	1	0	1	0	1
inactivo	0	1	0	0	0	1	0
suspendido	0	0	0	1	0	0	0

Combinar condiciones con AND/OR sobre bits

WHERE ESTADO = 'activo' AND país='AR'

activo: 1010101

AND

país AR:1100101

Resultado: 1000101

filas:1, 5, 7

Operación AND bit a bit (sin tocar la tabla)

Índices en Base de Datos

Índices Bitmap

Ejemplos de cómo se crean los índices bitmap

Oracle (Bitmap nativo)

```
CREATE BITMAP INDEX idx_bmp_estado ON clientes  
(estado);
```

PostgreSQL lo construye internamente en tiempo de ejecución no se crea con CREATE INDEX, el motor lo usa solo cuando conviene

```
SELECT * FROM clientes WHERE estado = 'activo' AND  
pais = 'AR';
```


Índices en Base de Datos

Índices Multicolumna (compuesto)

Índices en Base de Datos

Índices Multicolumna

Características

- Es una estructura que incluye **dos o más columnas** de una tabla
- Se usa cuando las consultas suelen aplicar filtros combinados sobre esas columnas.
- **Regla del prefijo:**
 - el índice solo se usa si la consulta incluye las columnas **en el mismo orden** en que fueron definidas.

Índices en Base de Datos

Índices Multicolumna

Ventajas

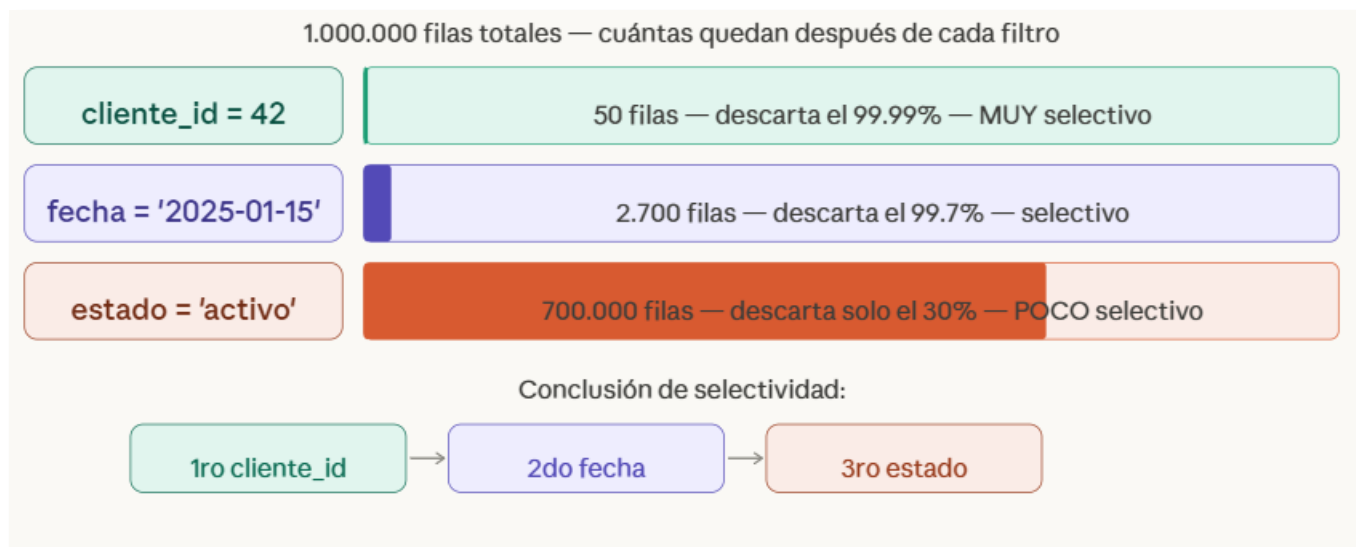
- Optimiza filtros combinados:
 - mejora el rendimiento cuando se usan varias columnas en **WHERE**, **JOIN**, **GROUP BY**, **ORDER BY**.
- Reduce el número de índices:
 - evita crear múltiples índices individuales.
- Mejora el plan de ejecución:
 - el motor puede usar un solo índice para resolver la consulta.

Índices en Base de Datos

Índices Multicolumna

El orden de las columnas al crear el índice es una decisión de diseño:

- coloca primero la columna que **más reduce** el conjunto de filas.
- Si son iguales en selectividad, pone primero la que **más aparece sola** en las consultas reales.
- En PostgreSQL se puede usar **pg_stat_statements** para ver cuáles son las consultas más frecuentes y diseñar el índice en base a eso.



Índices en Base de Datos

Índices Multicolumna

Ejemplo de como crear el índice y su uso

```
CREATE INDEX idx_pedidos ON pedidos (cliente_id, fecha, estado);
```

USA el índice (respeta el prefijo)

WHERE cliente_id = 5 (primera columna)

WHERE cliente_id = 5 AND fecha > '2025-01-01' (primera + segunda)

WHERE cliente_id = 5 AND fecha > '2025-01-01' AND estado = 'pagado' (las tres)

NO usa el índice

WHERE fecha > '2025-01-01' (saltea cliente_id)

WHERE estado = 'pagado' (saltea las dos primeras)

Índices en Base de Datos

Índices Parciales

Índices en Base de Datos

Índices Parciales

¿Qué es un índice parcial?

es un índice que **no se crea sobre todas las filas** de una tabla, sino **solo sobre un subconjunto** de ellas que cumplen una condición específica (un predicado **WHERE**)

“un índice parcial no acelera todo...
pero acelera muchísimo lo que realmente importa.”

Índices en Base de Datos

Índices Parciales

Indexar sólo el subconjunto de filas que se consulta frecuentemente

Tabla de pedidos – 1.000.000 filas

completado – 900.000 filas (90%)

pendiente – 100.000 filas (10%)

WHERE
→

completado – no se indexa

pendiente – sólo esto se indexa

Índice completo - desperdicio

```
CREATE INDEX idx ON pedidos (fecha)  
indexa 1.000.000 filas — el 90% nunca se consulta
```

```
CREATE INDEX idx_pendientes ON pedidos  
(fecha)WHERE estado = 'pendiente'
```


Índices en Base de Datos

Índices Parciales

CREATE INDEX idx ON pedidos (fecha)

indexa 1.000.000 filas — el 90% nunca se consulta

**CREATE INDEX idx_pendientes ON pedidos
(fecha) WHERE estado = 'pendiente'**

Comparación

Índice completo

1.000.000 entradas — pesado, lento de mantener

Índice parcial

100.000 entradas — 10x más pequeño y rápido

Las consultas:

WHERE estado = 'pendiente' AND fecha < NOW()

usan el índice parcial

WHERE estado = 'pendiente'

no lo usan (hacen un escaneo completo de la tabla)

Ejemplos: borrados, valores nulos o irrelevantes

Índices en Base de Datos

Índices Cubrientes

(Covering Index)

Índices en Base de Datos

Índices Cubrientes

¿Qué es un índice cubriente?

- es un índice que contiene **todas las columnas necesarias para resolver una consulta.**
- No hace falta ir a la tabla (evita I/O a la tabla).
- La consulta se resuelve **solo con el índice.**
- El índice **cubre** toda la consulta.

“El mejor acceso a disco... es el que no ocurre.”

Índices en Base de Datos

Índices Cubrientes

Consideraciones

- **Espacio en disco**
 - El índice es más grande porque almacena más columnas.
- **Mantenimiento**
 - Insertar/actualizar/borrar filas puede ser más costoso
- **Selección de columnas**
 - Conviene incluir solo las columnas críticas para la consulta.
 - Pocas columnas en SELECT

Índices en Base de Datos

Índices Cubrientes

Comparativa

Sin índice cubriente



Índice Cubriente



Índices en Base de Datos

Índices Cubrientes

Ejemplo de como crear el índice y su uso

Índice simple: sabe dónde están las filas, pero necesita ir a buscarlas

```
CREATE INDEX idx_simple ON pedidos (cliente_id);
```

Índice cubriente: incluye las columnas que necesita la consulta

```
CREATE INDEX idx_cubriente ON pedidos (cliente_id) INCLUDE (total, fecha);
```

Esta consulta nunca toca la tabla — Index Only Scan

```
SELECT total, fecha FROM pedidos WHERE cliente_id = 42;
```

Verificar en el EXPLAIN

```
EXPLAIN SELECT total, fecha FROM pedidos WHERE cliente_id = 42;
```

→ Index Only Scan using idx_cubriente

Índices en Base de Datos

Índices Funcionales

(Functional Indexes / Index on Expression)

Índices en Base de Datos

Índices Funcionales

Si buscamos usuarios por **email** pero las consultas usan:

LOWER(email) = 'juan@mail.com'

¿el índice tradicional sobre **email** nos sirve?

Índices en Base de Datos

Índices Funcionales

¿Qué es un índice funcional?

es un índice que se construye sobre el **resultado de una función o expresión** aplicada a una o más columnas, en lugar de sobre las columnas directamente.

Índices en Base de Datos

Índices Funcionales

¿Qué problema resuelve?

cuando la consulta **transforma** la columna antes de comparar, el B+Tree no puede ayudar porque el valor en el índice es distinto al que se está buscando.

... WHERE **LOWER**(email) = 'juan@mail.com'

Índices en Base de Datos

Índices Funcionales

Consideración

La función usada en el índice **debe ser determinista**:

siempre tiene que devolver el mismo resultado para el mismo input. **LOWER()**, **UPPER()**, **EXTRACT()** son deterministas.

NOW() o **RANDOM()** no lo son y no se pueden indexar.

Índices en Base de Datos

Índices Funcionales

¿Por qué un índice normal no alcanza?

ID	Email (en tabla)
1	Juan@MAIL.com
2	MARIA@mail.com
3	pedro@Mail.COM
4	ANA@MAIL.com

Índice normal sobre email

Guarda el valor tal cuál está en la tabla

ANA@MAIL.com -> ID 4

Juan@MAIL.com -> ID 1

MARIA@mail.com-> ID

pedro@Mail.COM-> ID 3

El problema:

...WHERE LOWER(email) = 'juan@mail.com' (transforma la columna)

El SGBD busca 'juan@mail.com' en el índice pero encuentra 'Juan@MAIL.com' (no coincide)

Resultado: ignora el índice y busca en toda la tabla en cada consulta

Índices en Base de Datos

Índices Funcionales

La solución con un índice funcional

el índice guarda el resultado de **LOWER(email)** (no el email original)

ana@mail.com → ID 4

juan@mail.com → ID 1

maria@mail.com → ID 2

pedro@mail.com → ID 3

Ahora **LOWER(email)** = 'juan@mail.com'
encuentra exactamente 'juan@mail.com'
en el índice

La clave es que el índice funcional **pre-computa** la expresión una vez al insertar,
y la consulta la encuentra lista

La regla es simple: lo que está dentro de **LOWER()** en la consulta debe ser exactamente lo mismo que está dentro de **LOWER()** en el índice.

Índices en Base de Datos

Índices Funcionales

Ejemplo

Sin índice funcional → Recorre toda la tabla

```
CREATE INDEX idx_email ON usuarios (email);  
WHERE LOWER(email) = 'juan@mail.com'; (NO usa el índice)
```

Con índice funcional → usa el índice

```
CREATE INDEX idx_email_lower ON usuarios (LOWER(email));  
WHERE LOWER(email) = 'juan@mail.com'; (Index Scan directo)
```

Índices en Base de Datos

Índices Funcionales

Casos de uso más frecuentes en la práctica:

Búsqueda de texto case-insensitive

```
CREATE INDEX idx_nombre ON clientes (LOWER(nombre));  
WHERE LOWER(nombre) = 'martínez';
```

Extraer parte de una fecha

```
CREATE INDEX idx_anio ON ventas (EXTRACT(YEAR FROM fecha));  
WHERE EXTRACT(YEAR FROM fecha) = 2025;
```

Expresión matemática

```
CREATE INDEX idx_descuento ON productos ((precio * 0.9));  
WHERE (precio * 0.9) < 1000;
```

Convertir tipo

```
CREATE INDEX idx_codigo ON articulos (CAST(codigo AS INT));  
WHERE CAST(codigo AS INT) > 500;
```

Índices en Base de Datos

Índices BRIN (Block Range Index)

Índices en Base de Datos

Índices BRIN

¿Qué es un índice BRIN (Block Range Index)?

- En lugar de indexar fila por fila como el B+Tree, divide la tabla en **bloques de disco** y guarda solo el mínimo y máximo de cada bloque (PostgreSQL).
- Es el índice más liviano que existe (mucho menos espacio que un B-Tree).
- **Los datos deben estar correlacionados con su orden físico en disco.**
- Se usa para tablas muy grandes.

Índices en Base de Datos

Índices BRIN

Analogía

Es como el índice de un libro que dice:

"En las páginas 1-50 se habla de los temas **A a la M**"

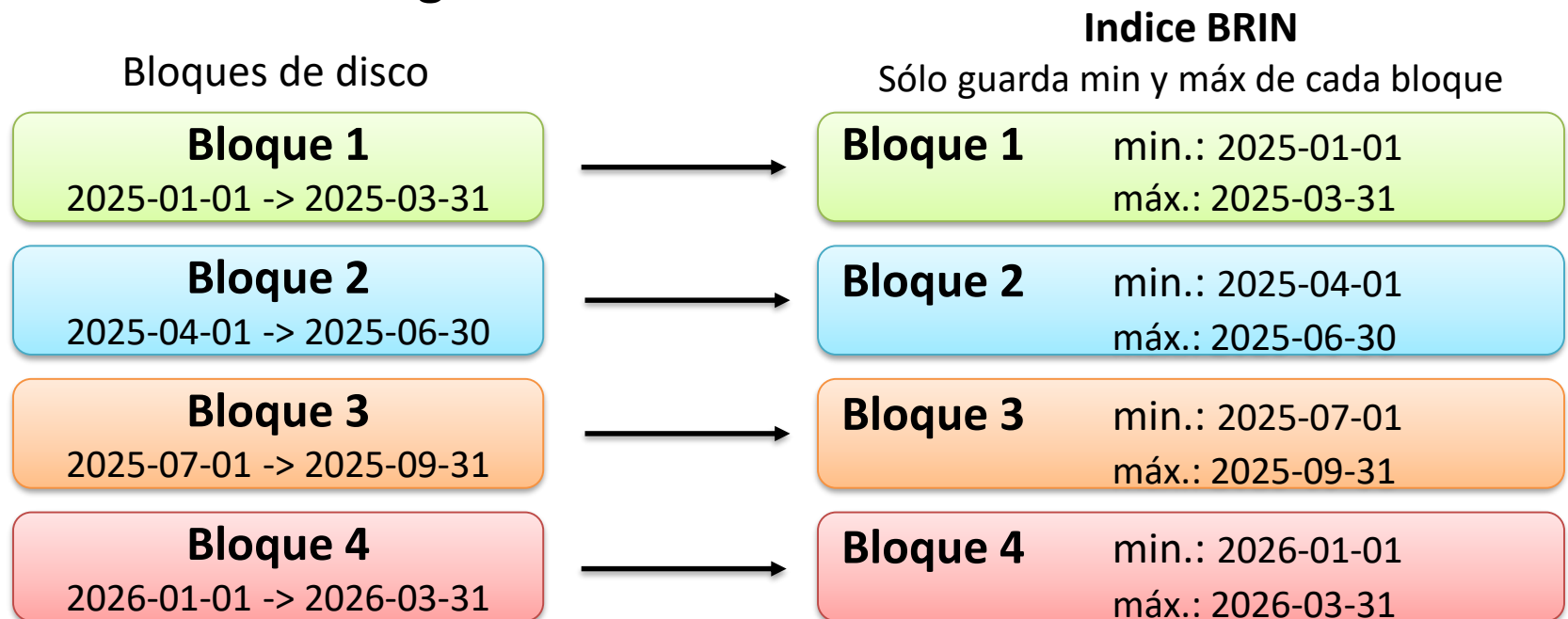
No se sabes exactamente dónde está algo, pero sí se sabe **en qué bloque de páginas** podría estar.

Índices en Base de Datos

Índices BRIN

Estructura del BRIN

Si tenemos una Tabla de **logs** de 500 millones de filas y con datos ordenados cronológicamente



El índice completo ocupa solo unos kilobytes (no importa cuántas filas tenga la tabla)

Índices en Base de Datos

Índices BRIN

¿Dónde está la información para acceder al registro de la tabla?

- NO hay una referencia directa.
- El índice BRIN sacrifica la precisión individual para ganar en espacio.
- La **información** no es un ctid¹ de una fila, sino un rango de números de bloque.
 - **B-Tree**: Índice → **ctid** de la fila.
 - **BRIN**: Índice (Resumen min/max) → (a través del RevMap² → ctid de la tupla de resumen → Número de bloque(s) de la tabla.

¹ column tuple ID (número de pagina y número de offset)

² Range Map estructura de traducción que actúa como un "directorío de páginas"

Índices en Base de Datos

Índices BRIN

Ejemplo

Los logs se insertan en orden cronológico → correlación perfecta con disco

B+Tree: ocupa ~40GB para esta tabla

```
CREATE INDEX idx_btree_fecha ON logs (fecha);
```

BRIN: ocupa ~100KB para la misma tabla

```
CREATE INDEX idx_brin_fecha ON logs USING BRIN (fecha);
```

Ambos sirven para la siguiente consulta

```
SELECT * FROM logs
```

```
WHERE fecha BETWEEN '2025-01-01' AND '2025-01-31';
```

El índice BRIN tiene dos ventajas enormes:

1. Ocupa 400.000x menos espacio en disco
2. Los INSERTs son casi instantáneos (solo actualiza un min/max)

Índices en Base de Datos

Índices BRIN

¿Cuándo usar BRIN?

- **Logs y auditoría** (consultas por rango de fechas)
- **Datos de sensores** (series temporales)
- **Tablas muy grandes** (cientos de millones de filas)
- **Datos que se insertan en orden** (como un ID secuencial o timestamp)

¿Cuándo **NO** usar BRIN?

- Datos aleatorios (UUID, valores hash)
- Consultas puntuales (WHERE id = 123)
- Actualizaciones frecuentes que desordenan los datos
- Columnas sin correlación física con el orden de inserción

Índices en Base de Datos

Índices BRIN

Consideraciones prácticas

Que nos debemos plantear antes de crear cualquier índice

1. ¿La columna aparece en WHERE, JOIN u ORDER BY?
 - Si no, no vale la pena.
2. ¿La tabla tiene muchas filas (miles o más)?
 - Si es chica, el SGBD va a ignorar el índice de todas formas.
3. ¿La columna tiene alta cardinalidad (muchos valores distintos)?
 - Si todos los registros tienen el mismo valor, el índice no filtra nada útil.
4. ¿La tabla tiene muchas operaciones de escritura?
5. ¿La columna se usa frecuentemente?



Base de Datos Avanzada

